Final Report on Introduction to Computer Security, by Kirill N.

181472

February 23, 2026

My Slide

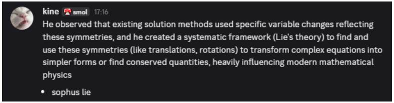

Figure 1: cat_speaking_wisdom

:You:

additional secure model stored on my local network can you please help me show my friend how to secure his network

THE AI Model:sec.intro:latest:

Okay, let's create a layered security model for your friend's local network, focusing on practical steps. I'll present this as if *you* are explaining it to your friend. I'll categorize it for clarity and include a "Threat Level" assessment for each step to show its impact. I'll also indicate the level of technical skill needed.

Important Disclaimer: No network is 100% secure. This model aims to significantly reduce risk, but diligence is key.

"Okay, so let's talk about securing your network. It's not about being paranoid, it's about being prepared. We'll build this in layers."

I. Basic Perimeter Security (Threat Level: Medium, Skill Level: Beginner)

Change the Default Router Password: *Seriously*. This is the biggest mistake people make. The default password is publicly known. (Skill: Beginner, Time: 5 minutes). "Your router has a default password, like 'admin' or 'password'. We *have* to change

:You:

how can i accept the ngrok access token into my AI client from remote workstation, with examples please

:sec.intro:latest:

Okay, let's break down how to securely accept an ngrok access token from a remote workstation into your AI client. This involves a combination of secure communication, token storage, and error handling. I'll present this with pseudocode and Python examples, as Python is often used for AI applications and scripting.

Core Concepts:

Secure Communication: Use SSH or a similar secure protocol to transfer the token. Avoid sending the token in plain text over the network.

Key Exchange (Optional but Recommended): For added security, use key exchange to establish a secure connection before transferring the token.

Token Storage: Store the token securely in a configuration file, environment variable, or dedicated secret management system. *Never* hardcode the token directly into your AI client's code.

:You:

we wish to imprint the device access token

:qwen2.5-coder:14b:

Imprinting a device access token involves securely embedding or storing it on a device so that it can be used for authentication and authorization purposes. The goal is to ensure that the token is both secure and easily accessible by your application. Below are various methods to achieve this, each with its own advantages and trade-offs.

1. Environment Variables

Advantages: - Secure (not hardcoded in code). - Easy to manage. - Works across different environments (development, staging, production).

Disadvantages: - Not persistent if the environment is restarted or changes. - Limited to single-token use unless managed with scripts.

Example Implementation:

```
# Set environment variable (Linux/Mac)
export DEVICE_ACCESS_TOKEN="your_token_here"

# Use in your application (Python example)
```

:You:

setting custom header on a msty connection or vscode continue dev ext

:qwen3-vl:8b:

:You:

please continue

:gemma3:27b:

Okay, let's dive into setting custom headers with `msty` connections and VS Code's Remote - SSH extension. There are a couple of different methods depending on *where* you need the headers applied and the context of the connection. I'll cover the most common scenarios and provide detailed instructions.

Understanding the Context

`msty` (Microsoft Telemetry Service): `msty` is often used to establish tunnels and connections to remote machines (often for debugging and development). Setting headers here influences the requests made *through* that tunnel. It's less common to customize headers at the `msty` level directly; it's more likely the *application* running through the tunnel handles the headers.

VS Code Remote - SSH: This extension lets you connect to a remote machine via SSH and develop as if you were working locally. Setting headers here primarily impacts the SSH connection itself and any subsequent requests made *through* that connection (e.g., if

:You:

share with friends

:gemma3:27b:

Okay, here are a few ways to share this information with your friends, depending on how you want to do it! I'll give you a few options, from simple copy/paste to more shareable formats. I'll break it down by method. I'll also include a summary you can easily copy/paste.

1. Copy/Paste the Text (Quickest)

Just select the text of our conversation and copy it (Ctrl+C or Cmd+C). Then paste it into:

Email: The simplest method.

Messaging App: (WhatsApp, Signal, Telegram, Messenger, Slack, Discord, etc.) Paste it directly into a chat. Some apps may have character limits, so you might need to break it up into multiple messages.

2. Create a Sharable Document (More Organized)

Google Docs: